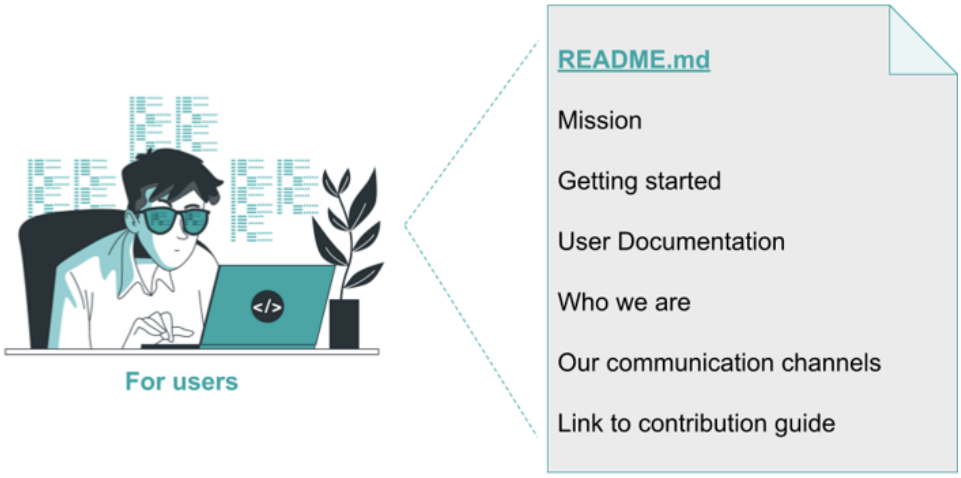
Documentação Base Padrão

README.md

Se ainda não existir, crie um arquivo `README.md` para o seu projeto.

Ele deve conter:

- Missão do projeto de forma concisa possível. Deve responder qual é o propósito do projeto e permitir que os contributors façam uma boa primeira avaliação se um recurso sugerido provavelmente estará dentro do escopo do projeto ou não.
- Uma seção "Primeiros passos" para os usuários downstream do projeto. Deve explicar como configurar/integrar os artefatos do projeto, bem como uma explicação dos primeiros passos típicos para usuários iniciantes.
- Documentação mais detalhada para usuários do projeto - ou um link para isso.
- Documentação necessária para fazer modificações no projeto - ou um link para isso.
- Documentação sobre como contribuir para o projeto - ou um link para isso.
- Uma seção "Iniciando" que explique quais canais públicos, arquivados e vinculáveis de comunicação o projeto utiliza. Isso deve incluir um link para o rastreador de problemas do projeto, mas também para quaisquer outras mídias de discussão utilizadas.
- Uma seção "Quem somos" explicando quem são os Trusted Committers por trás do projeto - com uma explicação de que, em vez de entrar em contato com essas pessoas em particular, os canais públicos de comunicação acima devem ser usados para comunicação.
- Uma explicação dos critérios para o projeto transformar contributors em Trusted Committers - se esse caminho existir.



README.md

CONTRIBUTING.md

Se a explicação dos passos para fazer uma contribuição for muito complexa, crie um documento separado `CONTRIBUTING.md`. Este documento deve responder a perguntas frequentes que os contribuidores fizeram no passado. Não é necessário fornecer um livro abrangente de uma vez. Em vez disso, compartilhe informações que se mostraram necessárias para os contribuidores.

Provavelmente, isso abordará um ou mais dos seguintes tópicos:

- Como fazer checkout do código fonte do projeto do controle de versão.
- Como fazer modificações no projeto (potencialmente incluindo informações sobre diretrizes de codificação).
- Como fazer build do projeto.
- Como executar testes para garantir que as modificações acima não estejam introduzindo novos bugs.
- Como enviar suas modificações de volta ao projeto.
- Algumas informações sobre o tempo de resposta esperado para as modificações feitas.

CONTRIBUTING.md

Existem muitos bons exemplos de como escrever um arquivo `README.md` e que tipo de informações incluir em um arquivo `CONTRIBUTING.md` em vários projetos de código aberto. Páginas como como escrever um readme que se destaca, o Guia de Código Aberto do GitHub e o livro Producing Open Source têm informações valiosas sobre que tipo de informações fornecer.

Embora Producing Open Source não tenha um capítulo sobre como escrever um bom README em si, o capítulo Começando a partir do que você tem fornece uma lista bastante extensa de coisas que os membros da equipe, usuários e contributors precisarão. Os projetos InnerSource provavelmente não cobrirão todos esses aspectos desde o início, mas a própria lista é útil para inspiração do que pode ser abordado.

Além disso, este padrão vem com dois modelos básicos para você começar imediatamente: README-template.md e CONTRIBUTING-template.md

COMMUNICATION.md

COMMUNICATION.md

Se ainda não existir, crie um arquivo `COMMUNICATION.md` para o seu projeto.

Ele deve conter:

1. **Canais de Comunicação**
   Detalhe os canais públicos onde a comunicação do projeto acontece, com links diretos. No caso da Stone, inclua:
   - **Slack**: Liste os canais específicos que o projeto utiliza, como `#projeto-x` ou `#dev-tools`. Inclua orientações sobre como se juntar a esses canais (se necessário).
   - **GitHub Discussions**: Explique como e quando usar o espaço de discussões no GitHub, destacando que ele é ideal para propostas de novas ideias, perguntas abertas e debates colaborativos.
2. **Objetivo da Comunicação**
   Explique o propósito do arquivo: garantir que todos saibam como e onde se comunicar sobre o projeto, promovendo transparência, colaboração e rastreabilidade.
3. **Tipos de Comunicação e Onde Ocorrer**
   - **Dúvidas gerais**: Direcione os participantes ao canal de Slack principal ou à seção de discussões no GitHub.
   - **Reportar problemas**: Indique o uso da aba de Issues no GitHub, com um link para o repositório relevante.
   - **Pull Requests**: Explique que as discussões específicas a um PR devem ocorrer nos PRs diretamente no GitHub. Não espalhe a discussão sobre uma alteração.
   - **Sugestões de melhorias**: Oriente o uso do GitHub Discussions para ideias e melhorias, promovendo debates antes da formalização em Issues ou PRs.
4. **Frequência e Expectativas**
   - Defina prazos esperados de resposta para diferentes tipos de interação (exemplo: "Dúvidas no Slack serão respondidas em até X horas/dias úteis").
   - Informe sobre a periodicidade de revisões de Issues e PRs no GitHub (exemplo: "Pull Requests são revisados semanalmente às quartas-feiras").
5. **Histórico e Transparência**
   - Explique onde o histórico de discussões pode ser encontrado, como arquivos de Slack (se aplicável) ou repositórios de decisões documentadas no GitHub Discussions.
   - Inclua links para qualquer documento ou log relacionado.
6. **Contato com Mantenedores**
   - Explique como entrar em contato com os mantenedores do projeto para assuntos excepcionais.
   - Reforce que a comunicação deve priorizar os canais públicos para beneficiar toda a comunidade.

Exemplos na org stone-payments

Main Flipt

stone-payments/main-flipt/blob/main/README.md
github.com

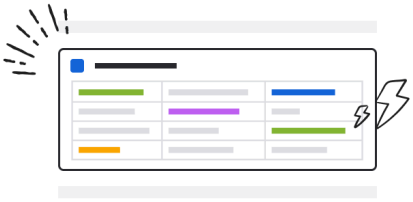
Turn your URLs into rich, interactive previews.
Saiba mais sobre os links inteligentes.

Conectar ao Github

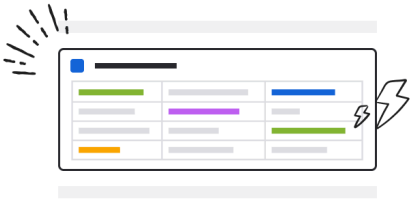
stone-payments/main-flipt/blob/main/CONTRIB...
github.com

Turn your URLs into rich, interactive previews.
Saiba mais sobre os links inteligentes.

Conectar ao Github



stone-payments/main-flipt/blob/main/COMMUN...
github.com

Turn your URLs into rich, interactive previews.
Saiba mais sobre os links inteligentes.

Conectar ao Github